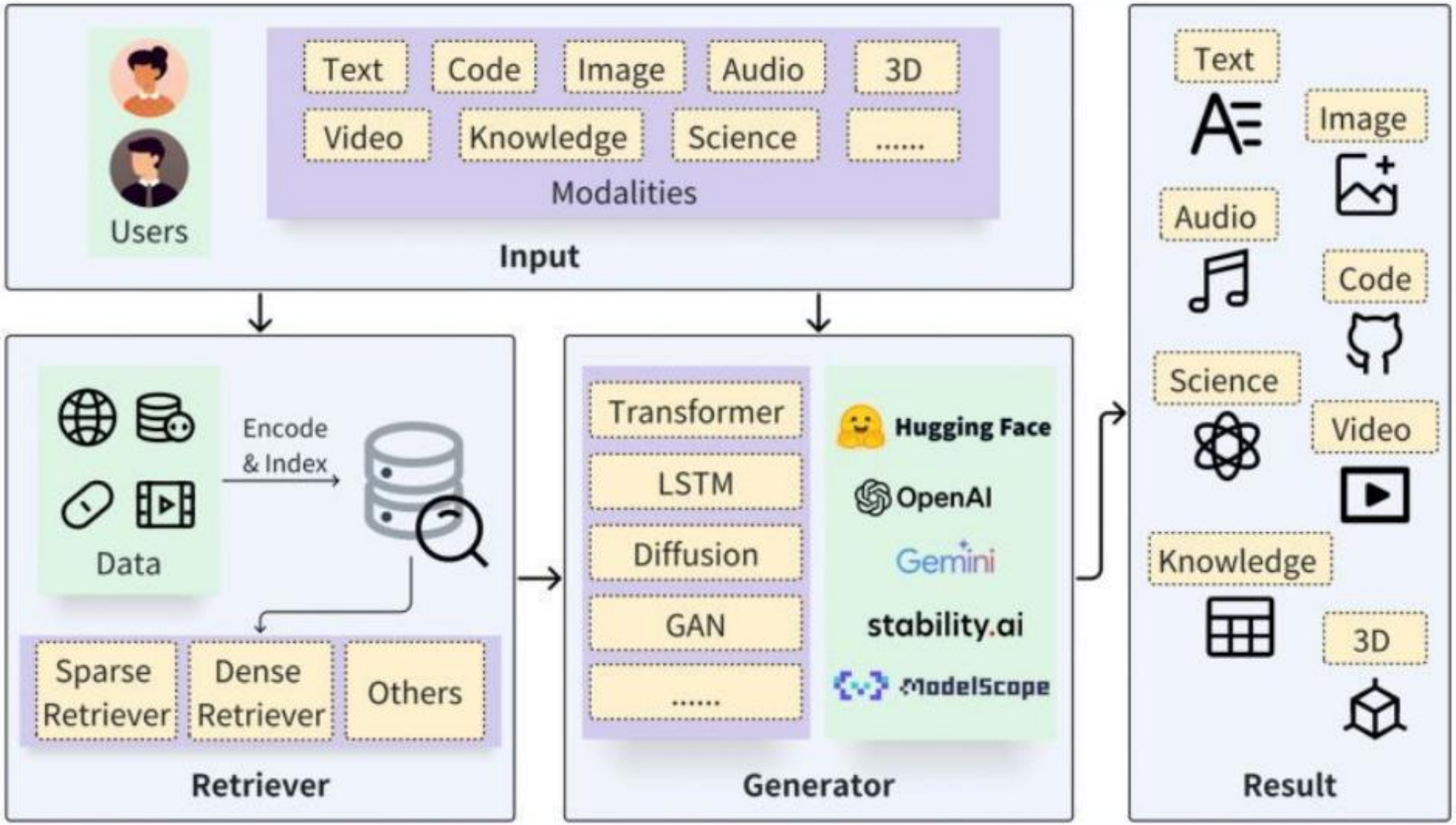
Retrieval Augmented Generation RAG

What is RAG

- Retrieval-Augmented Generation (RAG) is a natural language processing framework that enhances large language models (LLMs) by integrating external data retrieval with text generation. It retrieves relevant information from external sources/databases/custom source to improve response accuracy and relevance, mitigating issues like misinformation and outdated knowledge in generated content. So, RAG basically reduces the LLM hallucinations by providing contextually relevant responses through the data sources provided/attached

RAG Components



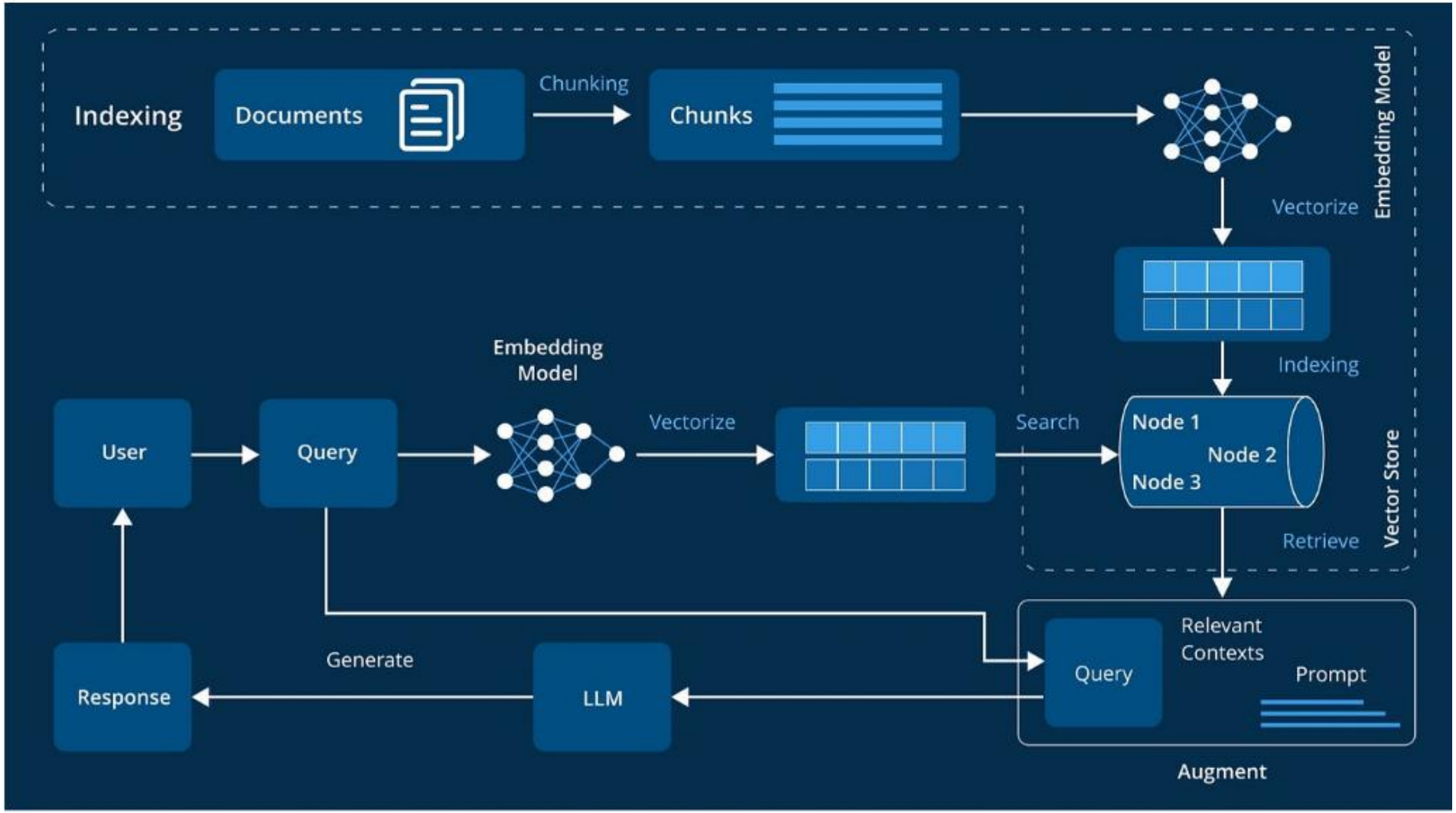
RAG pipeline

- **Retrieval**: This component helps you fetch the relevant information from the external knowledge base like a vector database for any given user query. This component is very crucial as this is the first step in curating the meaningful and contextually correct responses.
- **Augmentation**: This part involves enhancing and adding more relevant context to the retrieved response for the user query.
- **Generation**: Finally, a final output is presented to the user with the help of a large language model (LLM). The LLM uses its own knowledge and the provided context and comes up with an apt response to the user's query

Advantages of Retrieval Augmented Generation

- **Scalability**. RAG approach helps you with scale models by simply updating or adding external/custom data to your external database (vector database).
- **Memory efficiency**. Traditional models like GPT have limits when it comes to pulling fresh and updated information and fails to be memory efficient. RAG leverages external databases like a vector database — allowing it to pull in fresh, updated or detailed information when needed with speed.
- **Flexibility**. By updating or expanding the external knowledge source, you can adapt RAG to build any AI applications with flexibility.

RAG Architecture



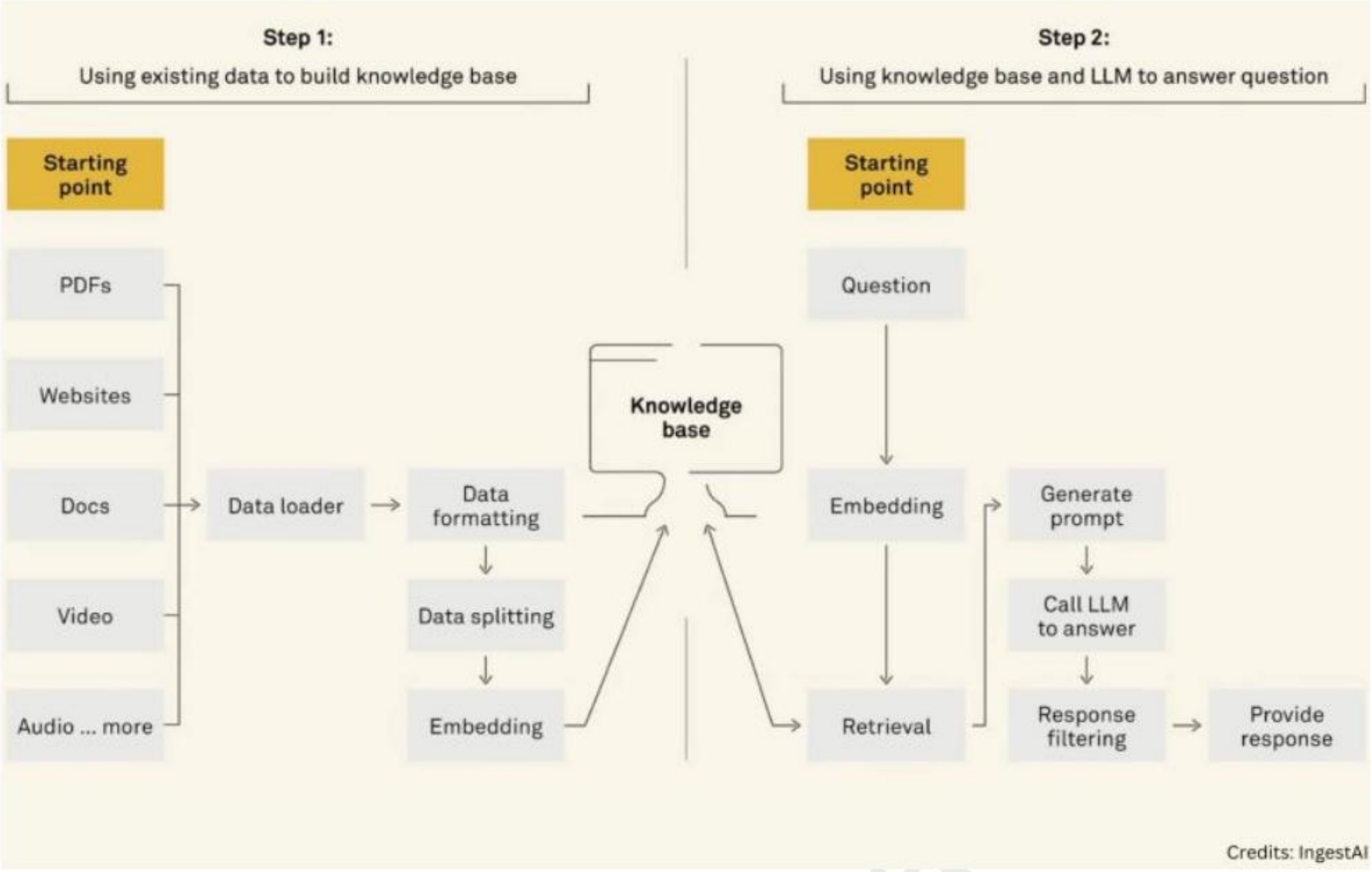
Important key points

- 1. Quality of External Knowledge Source: The quality and relevance of the external knowledge source used for retrieval are crucial.
- 2. Embedding Model: The choice of the embedding model used for retrieving relevant documents or passages from the knowledge source is important.
- 3. Chunk Size and Retrieval Strategy: Experiment with different chunk sizes to find the optimal length for context retrieval. Larger chunks may provide more context but could also introduce irrelevant information. Smaller chunks may focus on specific details but might lack broader context.

Important key points

- 4. Integration with Language Model: The way the retrieved information is integrated with the language model's generation process is crucial. Techniques like cross-attention or memory-augmented architectures can be used to effectively incorporate the retrieved information into the model's output.
- 5. Evaluation and Fine-tuning: Evaluating the performance of the RAG model on relevant datasets and tasks is important to identify areas for improvement. Fine-tuning the RAG model on domain-specific or task-specific data can further enhance its performance.
- 6. Ethical Considerations: Ensure that the external knowledge source is unbiased and does not contain offensive or misleading information.
- 7. Handling Out-of-Date or Incorrect Information: It's important to have strategies in place for handling situations where the retrieved information is out-of-date or incorrect.

RAG Retrieval Sources



Common data types

- Unstructured Data (Text): This includes plain text documents, web pages, and other free-form textual sources.
- Semi-Structured Data (PDF): PDF documents, such as research papers, reports, and manuals, contain a mix of textual and structural information.
- Structured Data (Knowledge Graphs): Knowledge graphs, such as Wikipedia and Freebase, represent information in a structured and interconnected format.
- LLM-Generated Content: Recent advancements have shown that LLMs themselves can generate high-quality content that can be used as a retrieval source. This approach leverages the knowledge captured within the LLM's parameters to generate relevant information.

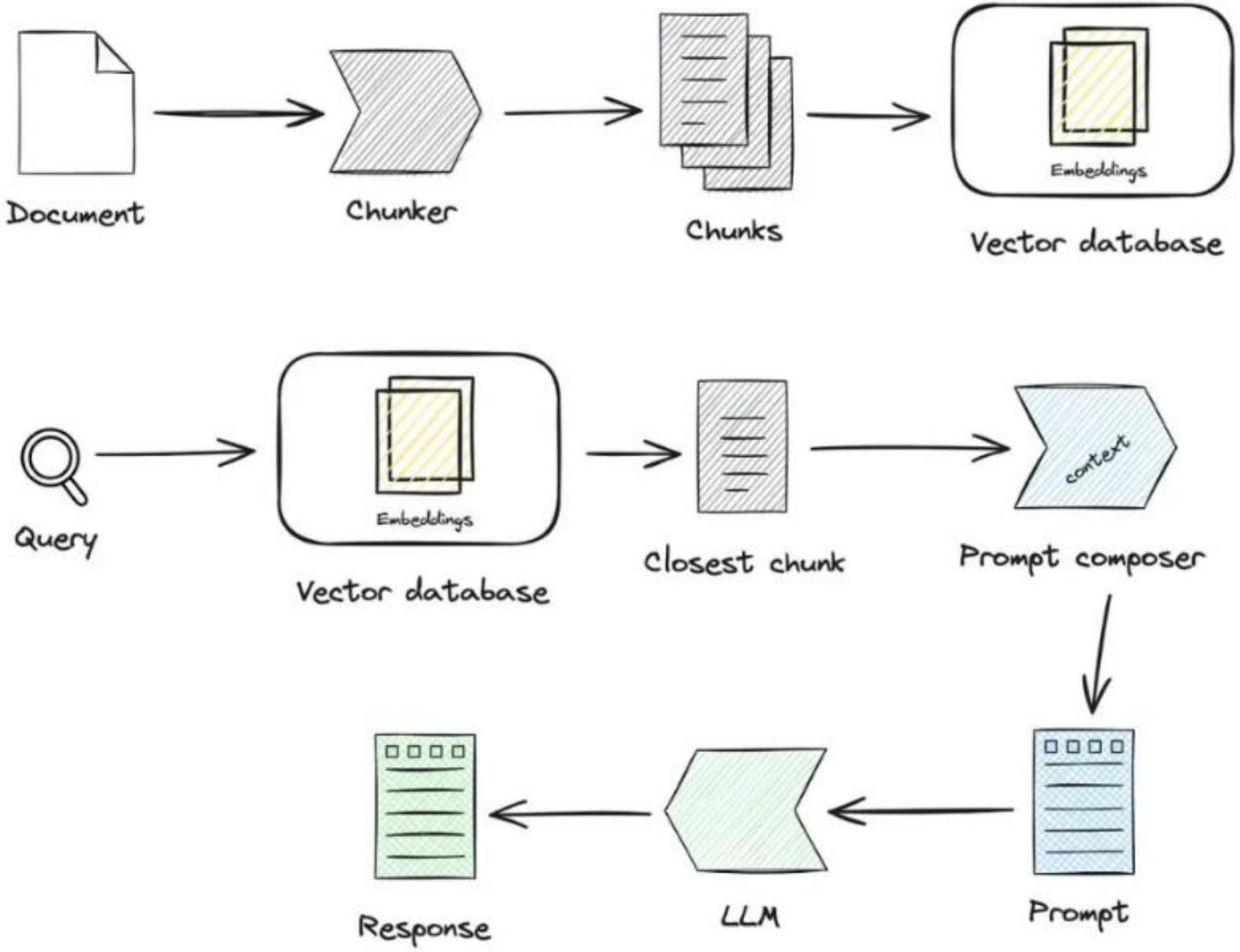
Common levels of retrieval granularity

- Phrase-Level Retrieval: This involves retrieving short phrases or snippets of text that are highly relevant to the query. Phrase-level retrieval can provide precise and targeted information but may lack broader context.
- Sentence-Level Retrieval: Sentence-level retrieval focuses on retrieving complete sentences that contain relevant information. It strikes a balance between specificity and context, making it suitable for a wide range of tasks.
- Chunk-Level Retrieval: Chunk-level retrieval involves retrieving larger chunks of text, such as paragraphs or sections. It provides more comprehensive information and context but may introduce noise and irrelevant details.
- Document-Level Retrieval: Document-level retrieval retrieves entire documents that are relevant to the query. While it offers the most extensive context, it may require additional processing to extract the most pertinent information.

Chunking Strategies in RAG

Chunking is the method of breaking down the large files into more manageable segments/chunks so the LLM applications can get proper context and the retrieval can be easy.

Chunking Strategies in RAG



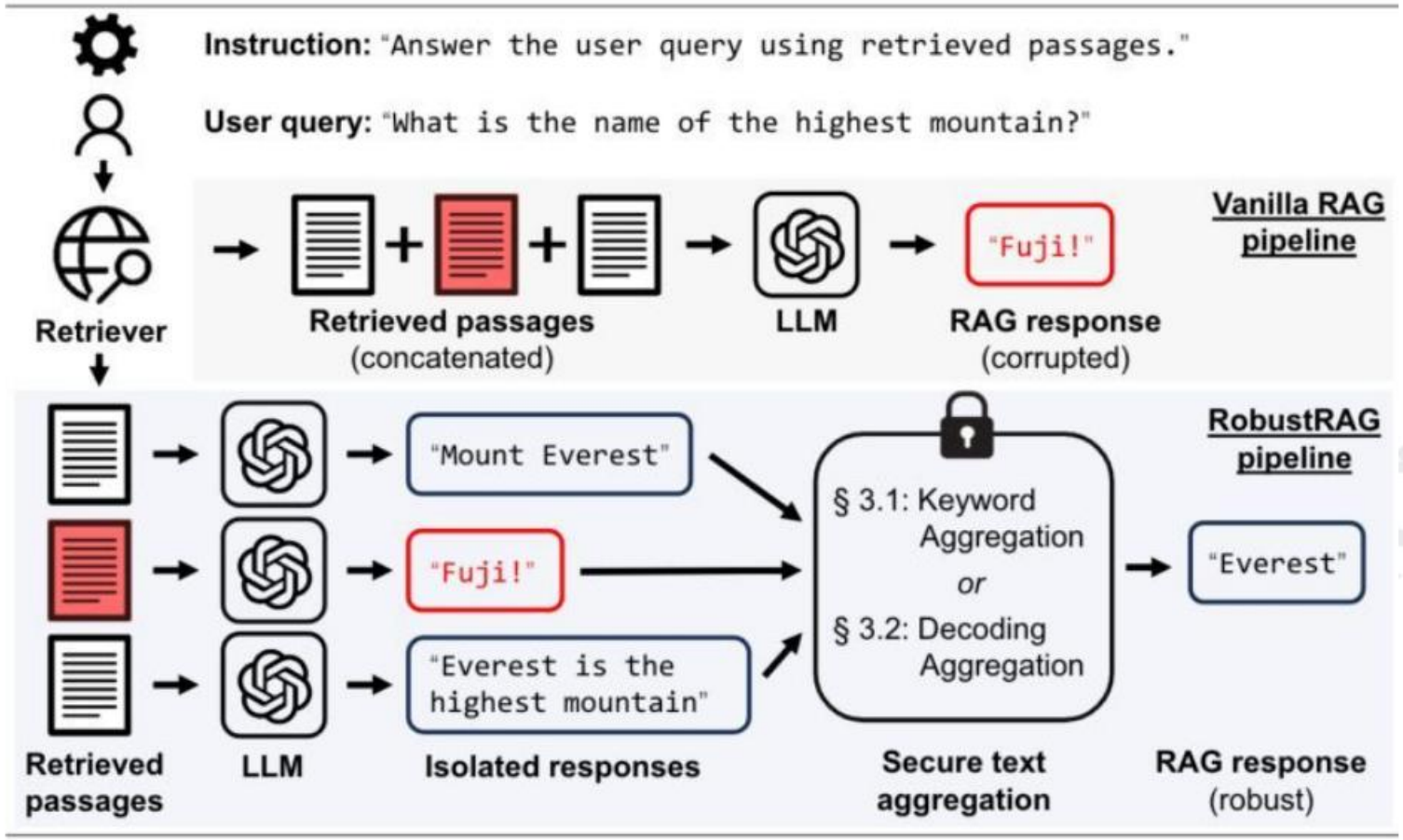
Chunking Strategies in RAG

- **Level 1, Fixed Size Chunking :** This is the most crude and simplest method of segmenting the text. It breaks down the text into chunks of a specified number of characters, regardless of their content or structure.
- **Level 2, Recursive Chunking :** While Fixed size chunking is easier to implement, it doesn’t consider the structure of text. Recursive chunking offers an alternative. In this method, we divide the text into smaller chunk in a hierarchical and iterative manner using a set of separators. Langchain framework offers RecursiveCharacterTextSplitter class, which splits text using default separators (“\n\n”, “\n”, “”,””)
- **Level 3, Document Based Chunking :** In this chunking method, we split a document based on its inherent structure. This approach considers the flow and structure of content but may not be as effective documents lacking clear structure.

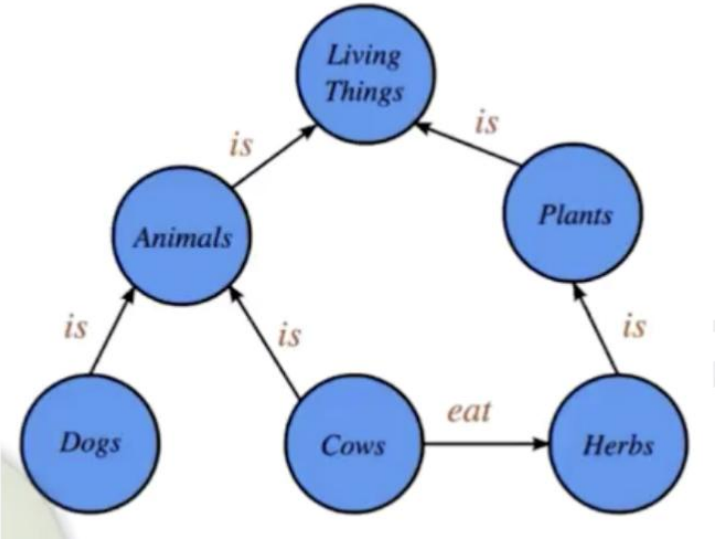
Chunking Strategies in RAG

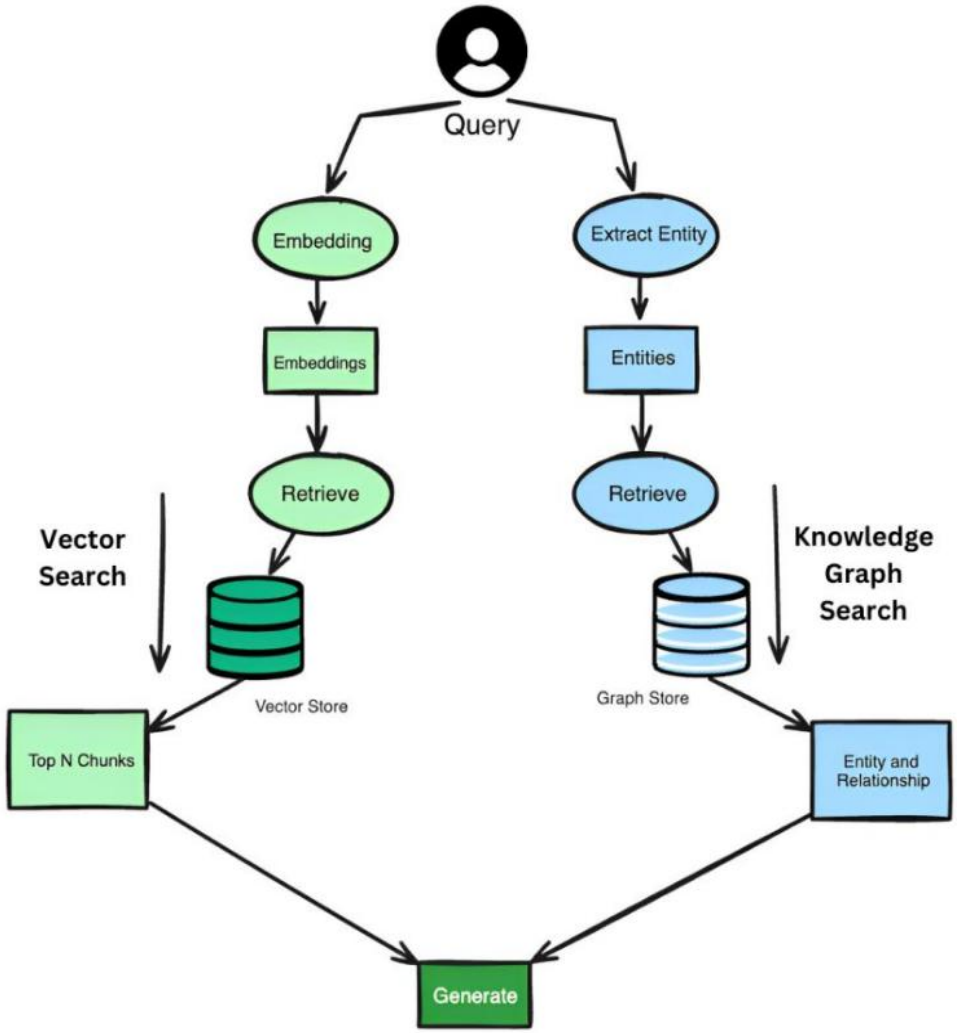
- **Level 4, Semantic Chunking :** All above three levels deals with content and structure of documents and necessitate maintaining constant value of chunk size. This chunking method aims to extract semantic meaning from embeddings and then assess the semantic relationship between these chunks. The core idea is to keep together chunks that are semantic similar.Llamindex has SemanticSplitterNodeParse class that allows to split the document into chunks using contextual relationship between chunks.
- **Level 5, Agentic Chunking** : This chunking strategy explore the possibility to use LLM to determine how much and what text should be included in a chunk based on the context.

Robust and Safe RAG

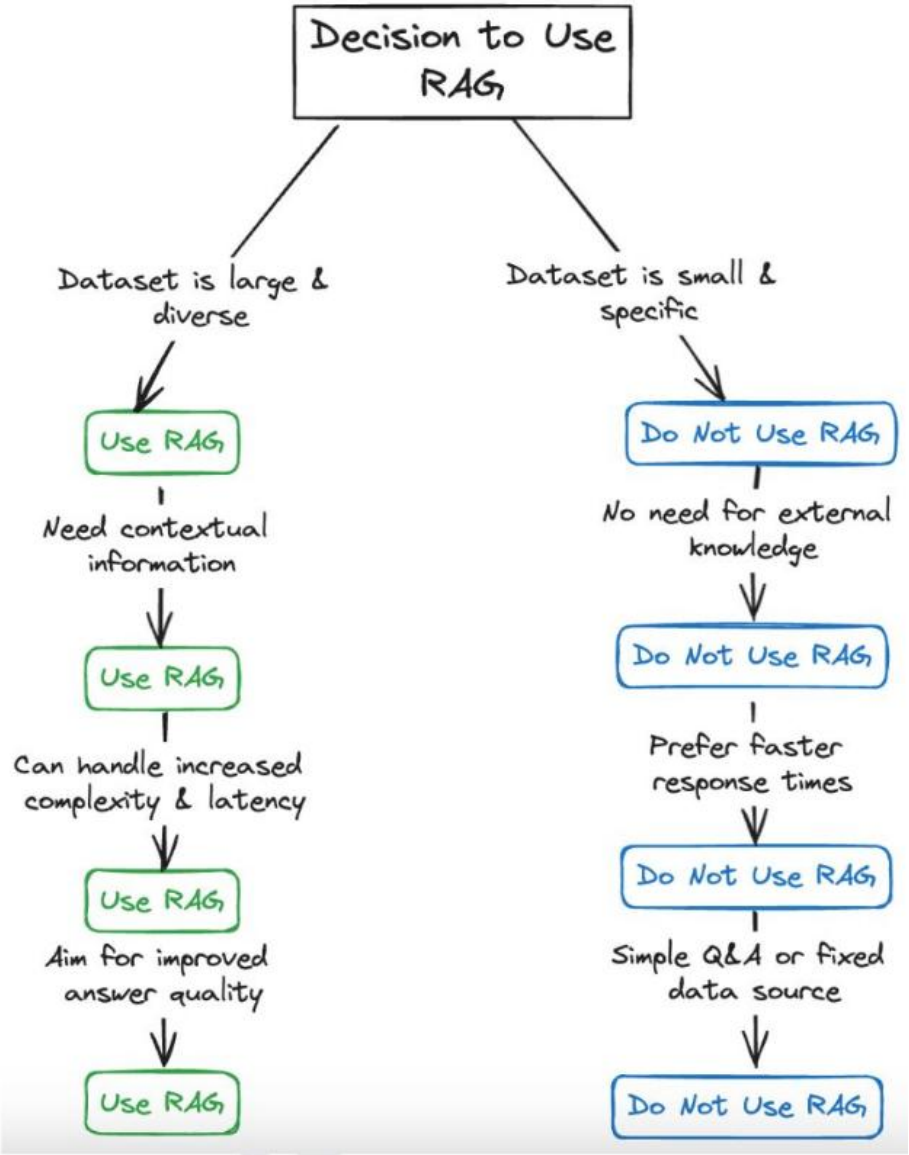
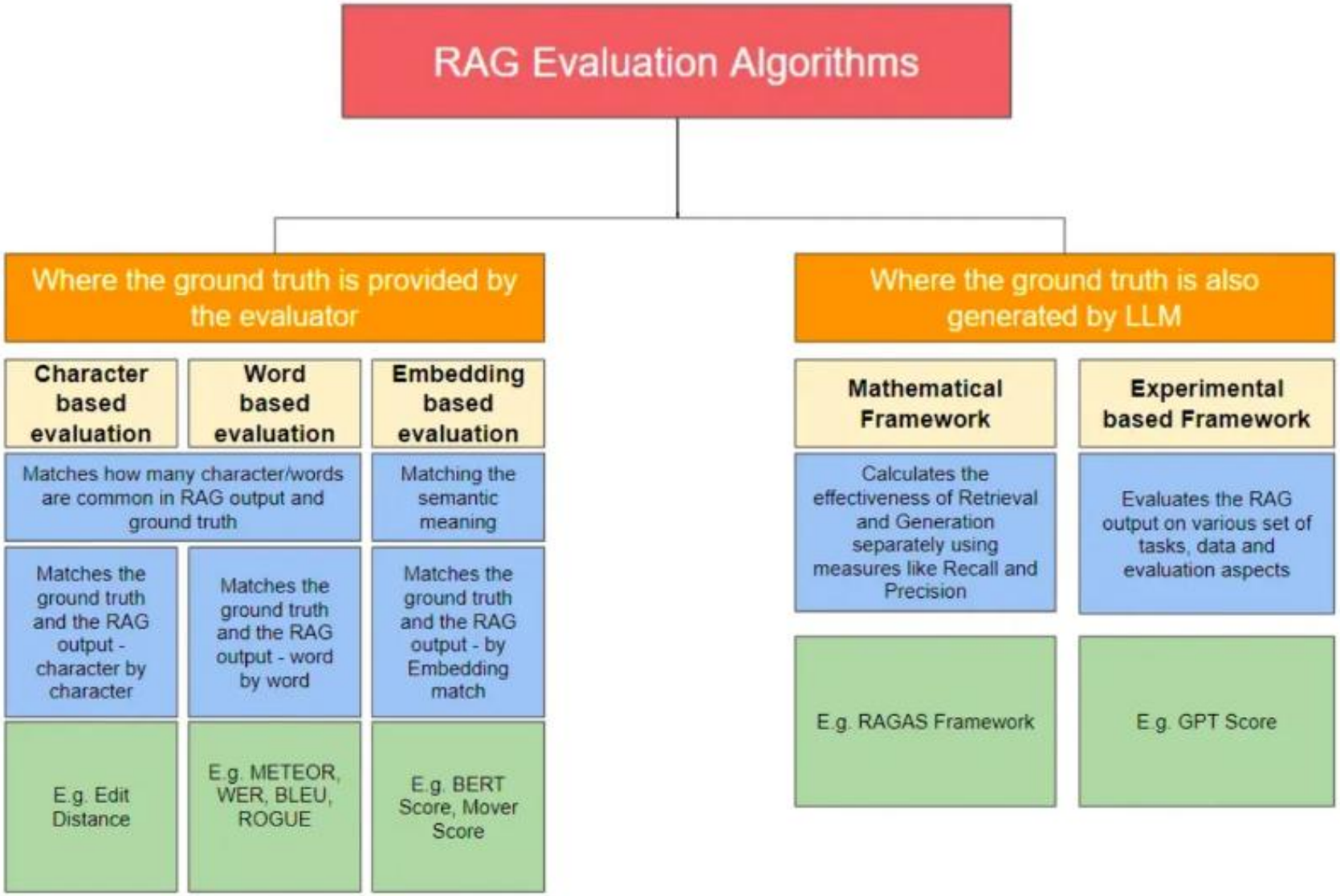


Graph RAG


Living Things
is
is
Animals
Plants
is
is
is
eat
Dogs
Cows
Herbs


Query
Embedding
Extract Entity
Embeddings
Entities
Retrieve
Retrieve
Vector Search
Knowledge Graph Search
Vector Store
Graph Store
Top N Chunks
Entity and Relationship
Generate

When to use RAG



Evaluation of RAG